MalaCall — Analysis Pipeline Manual

MalaCall — analysis & reporting pipeline

MalaCall is the end-to-end analysis for the 289-amplicon pan-*Plasmodium* panel designed by **MalaForge**. From raw reads it produces, **per sample and attributed per species**: species detection & ID (all 6 taxa incl. mixed infections), drug-resistance genotypes with drug interpretation, diagnostic-resistance (*pfhrp2/3* deletions) and gene amplification (CNV), parasite diversity (microhaplotypes → complexity-of-infection & heterozygosity), and *P. knowlesi* subpopulation (Mf/Mn/cluster-3) — plus a **cohort surveillance summary**.

MalaCall is the pan-*Plasmodium* successor to the original Pf/Pv `malaforge` analysis pipeline: 6 taxa instead of 2, per-species attribution in co-infections, reference-is-mutant polarity handling, CNV + deletion + diversity + Pk-cluster in one run. (The `malaforge` name now belongs to the panel-**design** toolkit.)

> **Status:** in-silico designed and **validated against simulated ground truth (62/62 assertions across all modules incl. MLE-COI & relatedness — see "Validation")**. Research pipeline; wet-lab validation pending. Not for clinical use.

---

1. Pipeline at a glance

```
FASTQ ─▶ MAP ─▶ SPECIES_ID ─┬─▶ RESTANCE ──▶ resistance.tsv + drugs.tsv
 (per      │        │        ├─▶ CNV_DELETION ─▶ cnv.tsv (amplification + hrp2/3 deletion)
  sample)  │        │        ├─▶ DIVERSITY ──▶ diversity.tsv + microhap.tsv (COI, He)
           │        │        └─▶ PK_CLUSTER ──▶ cluster.tsv (Mf / Mn / cluster-3)   [if Pk]
           │        │
           ▼        ▼
        BAM     species.tsv ──────────────────▶ REPORT ─▶ <sample>.report.html
                                                  │
                                       all samples ─▶ COHORT ─▶ cohort.tsv + cohort.html
```

Stage	Tool	Does	Key output
DEMUX *(optional, ONT custom barcodes)*	demux.py	Split a pooled FASTQ by the panel's 24-nt barcodes (§4b)	per-sample `.fastq.gz`
`build_manifest` (once)	minimap2, in-silico PCR	Locate every amplicon on the pan-reference	`manifest/amplicons.tsv`
MAP	minimap2 + samtools	Align reads to pan-reference (6	sorted `.bam`

Stage	Tool	Does	Key output
		nuclear + 6 mito)	
SPECIES_ID	pysam	Tally mito `cox3` reads per taxon → species + mixed-infection call	`.species.tsv`
RESISTANCE	pysam	Genotype every marker codon **vs the sensitive allele**; drug interpretation	`.resistance.tsv`, `.drugs.tsv`
CNV_DELETION	pysam	Read-depth vs single-copy controls → amplification & *pfhrp2/3* deletion	`.cnv.tsv`
DIVERSITY	pysam, numpy	Microhaplotypes → per-locus alleles/He, within-host COI	`.diversity.tsv`, `.microhap.tsv`
PK_CLUSTER	pysam	Fixed cluster-diagnostic SNP alleles → Mf/Mn/cluster-3	`.cluster.tsv`
REPORT	python	Per-sample clinical HTML report	`.report.html`
COHORT	python	Surveillance dashboard over all samples	`cohort.tsv`, `cohort.html`
RELATEDNESS	python	Cohort MLE-COI + pairwise relatedness (+ McCOIL/Dcifer exports)	`coi.tsv`, `relatedness.tsv`

2. Requirements

- **Nextflow ≥ 23.04** (`java -version` ≥ 11) — for the production workflow.
- **minimap2**, **samtools** (≥ 1.15) on `PATH`.
- **Python 3.9+** with **pysam, numpy, pandas** (biopython for setup). One easy way:

```
conda create -n panplasmo -c bioconda -c conda-forge python=3.11
        pysam numpy pandas minimap2 samtools biopython
conda activate panplasmo
```

- **~1 GB RAM/sample**, a few CPU cores. Reference + panel ship with the repo (`refs/`, `targets/`, `design/`).

Install the CLI

From the repository root, install the `malacall` command (and its `malaforge` design sibling) into your env:

```
pip install -e .
malacall --version
malacall --help
```

Every command below is a `malacall <command>`. They wrap the underlying Nextflow workflow and per-stage Python tools, so you never have to call `nextflow` or `python bin/…` by hand (the raw equivalents are shown as *under the hood* notes if you want them). `malacall run` also sets `JAVA_HOME` and defaults the `localpy` profile for you.

> **What does `-profile localpy` do? (and how is CaptureForge involved?)** The run commands in this manual use `-profile localpy`. All this profile does is **point Nextflow at a local Python interpreter that has the pipeline's deps** (`pysam`, `numpy`, `pandas`, `biopython`) — one line in `nextflow.config`:
>
> ```
> localpy { params.python = '.../envs/<your-env>/bin/python3' }
>         // edit the path to your env
> ```
>
> Edit that path to your own environment (e.g. the `panplasmo` env from §2), and you're set. There is **no coupling to the CaptureForge software** — the analysis imports or invokes nothing from it.
>
> **Why you'll also see `-profile captureforge`:** it is a **retained alias** that resolves to the exact same thing. `captureforge` is the name of the conda env on the *development* machine, which was originally created for the panel-**design** work (that's where the CaptureForge/primer tooling was actually used); the analysis just reuses that env so a second one isn't needed. `-profile localpy` and `-profile captureforge` are interchangeable — prefer `localpy` for clarity.
>
> On any machine you have equivalent options, none tied to a specific env name: - `-profile localpy` (or the `captureforge` alias) — sets the Python interpreter to the path in `nextflow.config`, **or** - `--python /path/to/your/env/bin/python3` — point directly at any env with the deps (see §2 `conda create -n panplasmo …`), **or** - `-profile conda` (builds from `environment.yml`) / `-profile docker` (container) — fully self-contained, no local env needed.

---

3. One-time setup — build the amplicon manifest

Every stage needs to know where each amplicon sits on the pan-reference. Build it once:

```
malacall manifest
# -> analysis/manifest/amplicons.tsv  (+ .bed, amplicons.fa)
# under the hood: python3 analysis/bin/build_manifest.py
```

This locates all 289 amplicons on `refs/pan_reference.fa.gz` (extract by in-silico PCR → minimap2 for exact coordinates). Also generate the Pk cluster diagnostic panel once (ships pre-made; regenerate if the panel changes):

```
# analysis/refs/pk_diag_panel.tsv is provided; it maps each Pk
        diagnostic amplicon
# -> (offset, Mf-allele, Mn-allele, cluster3-allele). Populate from
        Divis 2020 fixed SNPs for production.
```

---

4. Input

A **samplesheet** CSV (`sample,fastq[,fastq2]`) — one row per sample. ONT/single-end omit `fastq2`:

```
sample,fastq
BF_2024_001,/data/reads/BF_2024_001.fastq.gz
```

```
BF_2024_002,/data/reads/BF_2024_002.fastq.gz
```

Illumina paired-end:

```
sample,fastq,fastq2
BF_2024_001,/data/BF_2024_001_R1.fastq.gz,/data/BF_2024_001_R2.fastq.gz
```

Reads are the barcoded, demultiplexed amplicon FASTQs from the two-step tailed-amplicon workflow (one file per sample after round-2 barcode demux).

Generate the samplesheet from a run folder — `make_samplesheet.py`

Writing the CSV by hand is error-prone, and every sequencer lays its output out differently. **`bin/make_samplesheet.py` takes a platform-native run folder and emits the ready-to-use samplesheet for you** — locating the FASTQs, deriving sample names, pairing R1/R2, merging split files, and skipping empties. See §4a below for full detail; the short form is:

```
malacall samplesheet --platform ont      --run-dir /data/minknow/RUN1
        --out samples.csv --barcode-map barcodes.csv
malacall samplesheet --platform illumina --run-dir
        /data/miseq/240830_M0/Fastq --out samples.csv
malacall samplesheet --platform mgi      --run-dir /data/mgi/E100xxx
        --out samples.csv
```

4a. Samplesheet generator — detailed reference

Why it exists

The pipeline consumes one flat CSV, but the three platforms deliver reads in three different shapes:

- **ONT / MinKNOW** demultiplexes into one *directory per barcode*, each containing many small FASTQ chunks.
- **Illumina** (bcl2fastq / BCL Convert) writes one file *per sample, per lane, per read* — `_R1_`/`_R2_`, `L001`/`L002`…
- **MGI (DNBSEQ)** writes paired `_1.fq.gz` / `_2.fq.gz` (sometimes `_R1`/`_R2`, sometimes per lane).

The generator normalises all three to the pipeline's `sample,fastq[,fastq2]` CSV.

Build / install

There is **nothing to build** — it is a single dependency-free Python 3 script (standard library only). Make it executable once and call it directly, or invoke via `python3`:

```
chmod +x analysis/bin/make_samplesheet.py
analysis/bin/make_samplesheet.py --help          # or: python3
        analysis/bin/make_samplesheet.py --help
```

The three run-folder layouts it understands

```
ONT (MinKNOW)                          Illumina (bcl2fastq/BCL Convert)
MGI (DNBSEQ)
RUN1/                                  240830_M0/Fastq/
E100xxx/
└── <sample_id>/                       ├── BF24-
001_S1_L001_R1_001.fastq.gz  ├── BF24_001_1.fq.gz
    └── fastq_pass/                    ├── BF24-
```

```
001_S1_L001_R2_001.fastq.gz  ├── BF24_001_2.fq.gz
        ├── barcode01/ *.fastq.gz     ├── BF24-
001_S1_L002_R1_001.fastq.gz  ├── BF24_002_1.fq.gz
        ├── barcode02/ *.fastq.gz     ├── BF24-
001_S1_L002_R2_001.fastq.gz  └── BF24_002_2.fq.gz
        ├── barcode03/ *.fastq.gz     ├── BF24-
002_S2_L001_R1_001.fastq.gz
        └── unclassified/ *.fastq.gz  └── BF24-
002_S2_L001_R2_001.fastq.gz
```

It searches the run folder **recursively**, so pointing it at the top-level run directory works for all three (it finds `fastq_pass/` for ONT, and the FASTQs wherever they sit for Illumina/MGI).

What it does, per platform

- **ONT:** finds `fastq_pass/`, treats each `barcodeNN` (and, with `--keep-unclassified`, `unclassified`) as one sample. All chunk FASTQs in a barcode are **merged** into one `<sample>.fastq.gz`. A run with no barcodes (`fastq_pass/*.fastq.gz` directly) becomes a single sample named after the run folder. Barcodes below `--min-bytes` (empty) are skipped.
- **Illumina / MGI (paired):** finds every R1 file (`_R1_001`, `_R1.`, `_R1_`, or `_1.fq/_1.fastq`), derives its R2 mate, and derives the **sample name** by stripping the Illumina/MGI tokens (`_S3`, `_L001`, `_R1`, `_001`, `_1`, extensions). Files that share a sample name across lanes are **merged** (all R1s → one file, all R2s → one file). An R1 with no R2 is emitted single-end with a warning.

Merging — how and why it's safe

When a sample maps to several files (ONT chunks, or lanes), they are **concatenated with a raw byte copy**. This is correct for gzip because *concatenated gzip streams are themselves a valid gzip stream* — so there is **no decompression/recompression**, it is fast and lossless. Merged files land in `--merge-dir` (default `<out_dir>/merged`). Single-file samples are **referenced in place** (never copied), so no disk is wasted.

Command-line reference

Flag	Required	Meaning
`--platform {ont,illumina,mgi}`	yes	Run-folder layout to parse.
`--run-dir PATH`	yes	The sequencer run folder (searched recursively).
`--out FILE`	yes	Samplesheet CSV to write.
`--merge-dir DIR`	no	Where merged FASTQs are written (default `<out_dir>/merged`).
`--barcode-map FILE`	no	ONT only: CSV `barcode,sample` to rename barcodes to real sample IDs.
`--min-bytes N`	no	Skip samples whose reads total < N bytes (default 1000) — drops empty barcodes.
`--keep-unclassified`	no	ONT: also emit the `unclassified` bin as a sample.

The `--barcode-map` file (ONT)

A two-column CSV mapping kit barcodes to your sample IDs (header optional):

```
barcode,sample
barcode01,BF24_001
barcode02,BF24_002
barcode03,BF24_003
```

Barcodes not listed keep their `barcodeNN` name. This is where you attach real specimen IDs from your run log.

Worked examples

```
# ONT, real sample IDs, keep the unclassified bin too, merged files
        into a scratch dir:
malacall samplesheet --platform ont --run-dir /data/minknow/BF_2024_08
        \
    --barcode-map run_log.csv --keep-unclassified --merge-dir
        /scratch/merged --out samples.csv

# Illumina, run straight on the BCL-Convert output folder:
malacall samplesheet --platform illumina --run-dir
        /seq/240830_M07123_0042/Alignment_1/Fastq \
    --out samples.csv

# ...then feed the pipeline:
malacall run --samplesheet samples.csv --outdir results --platform ont
```

Output

`sample,fastq` for ONT; `sample,fastq,fastq2` for Illumina/MGI. Paths are absolute; rows sorted by sample. A progress summary (n samples, merged location, any skipped/unpaired) prints to stderr. Example (ONT):

```
sample,fastq
BF24_001,/data/minknow/BF_2024_08/…/fastq_pass/barcode01/FAL_pass_0.fas
BF24_003,/scratch/merged/BF24_003.fastq.gz
```

Validated

The generator was tested against synthetic ONT, Illumina, and MGI run folders and the output fed straight into the pipeline: barcode renaming, multi-chunk/multi-lane merging, empty-barcode skipping, and R1/R2 pairing all behaved correctly, and a merged 2-chunk ONT barcode produced the expected combined read count end-to-end.

Troubleshooting

- *"No FASTQ samples found"* → wrong `--platform` or `--run-dir`; for ONT point at the folder that contains `fastq_pass/`. For an ONT run with reads directly in `fastq_pass/` (no barcodes), it becomes one sample.
- *A barcode is missing* → it was below `--min-bytes` (empty) or is `unclassified` (add `--keep-unclassified`).
- *Illumina samples split oddly* → non-standard filenames; the sample name is everything before `_S#/_R1`. Rename or post-edit the CSV if your naming is unusual.

4b. Demultiplexing the custom barcodes (ONT / PCR) — `demux.py`

Do you need this? It depends on how you barcoded:

- **Native ONT barcodes** (a single-index native kit): MinKNOW/dorado already demultiplexes into `fastq_pass/barcodeNN/` — you do **not** need this step; just run `make_samplesheet.py --platform ont` (§4a).
- **This panel's custom scheme** — the round-2 barcoding PCR appends **combinatorial dual 24-nt barcodes (96 × 96 = 9,216)**. MinKNOW's native demux does not know these barcodes, so the reads come off as **one pooled FASTQ** and must be demultiplexed here.

The barcode set

`refs/barcodes/barcodes_96.tsv` is a ready-to-order set of **96 orthogonal 24-nt barcodes** (minimum pairwise Hamming distance 10, GC 40–60 %, no homopolymer > 3) — safe for 2-mismatch demultiplexing. Regenerate or resize with:

```
python3 bin/generate_barcodes.py 96 24 10 >
        refs/barcodes/barcodes_96.tsv   # N length min-Hamming
```

For a real run, replace this with **your actual reagent barcode sequences** (same TSV format: `barcode_id<TAB>sequence`).

The sample → barcode map

A CSV telling demux which barcode pair identifies each sample (dual index):

```
sample,fwd_bc,rev_bc
BF24_001,BC001,BC009
BF24_002,BC002,BC010
```

(`fwd_bc`/`rev_bc` are IDs from the barcode TSV. For single-index runs use `--mode single` and omit `rev_bc`.)

How it works

`demux.py` anchors on the constant **CS1 / CS2** tags (robust to the variable adapter length in front of the barcode): the 24 nt immediately 5′ of **CS1** is the forward barcode; the 24 nt immediately 3′ of the reverse tag (revcomp **CS2**) is the reverse barcode. It tries **both read orientations** (ONT reads either strand), matches each barcode with mismatch tolerance, assigns the read only when its (fwd, rev) pair is in your map, and writes the **trimmed amplicon** (CS1…CS2 removed) — ready for mapping. Unmatched reads go to `unclassified.fastq.gz`.

Run it — standalone (also emits a samplesheet)

```
malacall demux \
  --fastq /data/minknow/RUN1/fastq_pass/*.fastq.gz \
  --barcodes refs/barcodes/barcodes_96.tsv \
  --sample-map map.csv \
  --outdir demuxed --mode dual --write-samplesheet
# -> demuxed/<sample>.fastq.gz, demuxed/unclassified.fastq.gz,
        demuxed/demux_stats.tsv, demuxed/samples.csv
malacall run --samplesheet demuxed/samples.csv --outdir results --
        platform ont
```

Run it — inside the pipeline (one command)

```
malacall run --demux \
  --pooled '/data/minknow/RUN1/fastq_pass/*.fastq.gz' \
  --barcode-map map.csv \
  --outdir results --platform ont
# DEMUX runs first, then MAP → … → REPORT → COHORT on the per-sample
        reads automatically.
```

Options: `--barcodes FILE` (default `refs/barcodes/barcodes_96.tsv`), `--demux_mode dual|single`, `--bc_mismatch N` (default 2). `demux_stats.tsv` reports reads-per-sample and the unclassified rate.

Validated

On a simulated pooled run (94,886 barcoded reads, both orientations, 2 % ONT error) demux assigned **96.4 %** of reads with **0 misassignments** (100 % precision), and the demultiplexed reads reproduced the full per-sample analysis identically to the pre-pooling truth.

5. Running the pipeline

5a. Production — Nextflow (recommended)

```
cd analysis

# ONT run, 8-plex demo. `malacall run` sets JAVA_HOME + the localpy
        profile for you
# (add --dry-run to print the nextflow command it builds; --profile/--
        python to override the env):
malacall run \
  --samplesheet samples.csv \
  --outdir results \
  --platform ont

# Illumina 2×250:
malacall run --samplesheet samples.csv --outdir results --platform
        illumina

# Built-in end-to-end self-test on bundled simulated data (62/62):
malacall validate
```

Useful params: `--min_frac 0.02` (species-call fraction), `--min_vaf 0.05` (minor resistant-clone VAF), `--python /path/to/python`, `--cohort_name my_survey`. Nextflow writes a **DAG, timeline, resource trace, and execution report** into `results/` automatically. Resume after interruption with `-resume`.

5b. Single sample — Python driver (no Nextflow)

```
malacall sample \
  --fastq /data/reads/BF_2024_001.fastq.gz \
  --sample BF_2024_001 \
  --outdir results \
  --platform ont
# -> results/BF_2024_001/BF_2024_001.
        {bam,species.tsv,resistance.tsv,drugs.tsv,cnv.tsv,diversity.tsv,cluster.tsv,repo
```

5c. Batch + cohort with the driver

```
for f in /data/reads/*.fastq.gz; do
  s=$(basename "$f" .fastq.gz)
  malacall sample --fastq "$f" --sample "$s" --outdir results --
        platform ont
done
malacall cohort results my_survey
# -> results/my_survey.cohort.tsv + results/my_survey.cohort.html
```

5d. Run any single stage (debugging / re-analysis)

```
# after MAP has produced the BAM:
BAM=results/BF_2024_001/BF_2024_001.bam ;
        M=analysis/manifest/amplicons.tsv
python3 analysis/bin/species_id.py        $BAM $M BF_2024_001
        out.species.tsv
python3 analysis/bin/resistance_genotype.py $BAM $M
        targets/codon_to_genomic_pan.tsv BF_2024_001 out/ --species
        Pf,Pv
python3 analysis/bin/cnv_deletion.py      $BAM $M BF_2024_001
        out.cnv.tsv --species Pf
python3 analysis/bin/diversity.py         $BAM $M BF_2024_001 out/ --
        species Pf
python3 analysis/bin/pk_cluster.py        $BAM $M
        analysis/refs/pk_diag_panel.tsv BF_2024_001 out.cluster.tsv
```

---

6. Outputs & how to read them

Per sample (`results/<sample>/`):

- **`<sample>.species.tsv`** — cox3 reads, proportion and present/absent per taxon. >1 present = **mixed infection**; proportions are the relative parasitaemia of each species.
- **`<sample>.resistance.tsv`** — one row per marker codon: `sensitive_aa`, `observed_aa`, `resistant_alleles(vaf)`, `depth`, `call` (RESISTANT / sensitive / genotype / no_call). **Calls are always made against the SENSITIVE allele**, so reference-is-mutant loci (e.g. *pmdhfr* 58/114, where the PmUG01 reference itself carries the mutant) are correct — a genuinely sensitive parasite is called *sensitive*.
- **`<sample>.drugs.tsv`** — per species per drug: "RESISTANCE-ASSOCIATED MUTATION" + the markers, or "no resistance marker detected".
- **`<sample>.cnv.tsv`** — gene amplification (copy ratio vs single-copy controls; `AMPLIFIED (N copies)`) and **RDT-escape deletions** (*pfhrp2/pfhrp3* `DELETED` when depth ≈ 0 vs controls).
- **`<sample>.diversity.tsv`** — per species: loci typed, polymorphic loci, mean He, and **COI** (complexity of infection). `<sample>.microhap.tsv` is the per-locus detail.
- **`<sample>.cluster.tsv`** — *P. knowlesi* subpopulation call (Mf / Mn / cluster-3) with per-locus votes.
- **`<sample>.report.html`** — the clinical one-pager combining all of the above.

Cohort (`results/`):

- **`<cohort>.cohort.tsv` / `.cohort.html`** — species distribution, per-drug resistance frequencies, *pfhrp2/3* deletion frequency, amplification frequency, COI distribution — a molecular-surveillance dashboard.
- **`<cohort>.coi.tsv`** — maximum-likelihood **COI** per sample per species.
- **`<cohort>.relatedness.tsv`** + **`<cohort>.related_pairs.tsv`** — pairwise **relatedness (r)** between samples (and the highly-related pairs, r ≥ 0.25) — for transmission-chain / outbreak analysis.
- **`export/`** — `dcifer_input.tsv` and `realmccoil_<species>.tsv`, ready for **Dcifer** and **THE REAL McCOIL** if you want the canonical published estimators.

---

7. Interpretation notes

- **Mixed infections:** each species is genotyped from its own species-specific amplicons, so a Pf+Pv co-infection yields independent Pf and Pv resistance/diversity calls. Poc+Pow is the one pair sharing a few amplicons (resolved for species ID by cox3).
- **Minor clones:** resistant alleles are reported with their VAF; `--min-vaf` sets the calling floor (default 5 %).
- **COI** appears twice: a quick per-sample heuristic in `<sample>.diversity.tsv`, and the **maximum-likelihood COI** across the cohort in `<cohort>.coi.tsv` (preferred). **Relatedness** (`<cohort>.relatedness.tsv`) is a frequency-adjusted IBS-moment estimate on population-informative microhaplotypes: r≈1 = clonal/highly related, r≈0 = unrelated. Run `malacall relatedness <results_dir> <cohort_name>` after the batch (the Nextflow workflow does this automatically), then optionally feed `results/export/` to Dcifer/McCOIL.
- **Discovery-grade markers** (non-Pf dhps/k13/crt/mdr1) have no validated codons; they are covered as amplicons and any non-synonymous variant is visible in the pileup, but no R/S call is asserted.

---

8. Validation (rigorous, reproducible)

The pipeline is validated against **simulated ground truth**: `test/simulate.py` builds reads encoding a known species mix, resistance genotypes (including reference-is-mutant polarity), CNV, gene deletions, COI, and Pk cluster; `test/validate.py` runs the pipeline and checks every output against truth.

```
BB=python3   # a python with pysam/numpy
$BB analysis/test/simulate.py analysis/manifest/amplicons.tsv
        refs/pan_reference.fa.gz \
      targets/codon_to_genomic_pan.tsv analysis/refs/pk_diag_panel.tsv
        analysis/test/sim
for f in analysis/test/sim/reads/*.fastq.gz; do s=$(basename $f
        .fastq.gz);
  malacall sample --fastq $f --sample $s --outdir analysis/test/out --
        platform ont; done
malacall validate   # simulate + run + score, all in one
```

Expected: **`VALIDATION: 62 PASS, 0 FAIL` — every module recovers ground truth on all 13 samples**, covering Pf multidrug (incl. pfk13 C580Y, pfcrt K76T, dhfr/dhps, pfmdr1 amplification, pfhrp2 deletion, COI 2), Pf-sensitive, Pf+Pv mixed, Pm resistant *and* Pm sensitive (the polarity stress test), Pk (Mn cluster), Poc+Pow mixed, a parasite-negative control, a **clonally-related pair** (relatedness r=1.0), and **MLE-COI** recovery.

---

9. Analysis reference data (in the repo)

- `refs/pan_reference.fa.gz` — 6 nuclear + 6 mito, species-tagged `sp|contig`.
- `targets/codon_to_genomic_pan.tsv` — 126 marker codons with sensitive allele (`wt_aa`), polarity (`ref_status`), and associated drugs.
- `analysis/manifest/amplicons.tsv` — amplicon → pan-reference coordinates (built in step 3).
- `analysis/refs/pk_diag_panel.tsv` — Pk cluster diagnostic SNP alleles.

10. Troubleshooting

- *All resistance calls* `no_call(depth<10)` → reads didn't map to the expected contig; check `samtools idxstats` and that the manifest matches the reference you mapped to.

- *A species missed in a mixed infection* → lower `--min-frac`; check cox3 read counts in `.species.tsv` (cox3 reads carry MAPQ 0 by design — they are counted at MAPQ 0).
- *Nextflow can't find python deps* → pass `-profile localpy` (the local env that has them — see the note in §2; **not** the CaptureForge software; `-profile captureforge` is a retained alias for the same thing) or `--python /path/to/env/bin/python3`, or use `-profile conda/docker`.

11. Known limitations

- Discovery-grade non-Pf resistance and diversity (see the design dossier §6/§8).
- Pk cluster calling needs the diagnostic-allele panel populated from the Divis-2020 fixed-SNP list for field use; the bundled panel is for validation.
- COI and pairwise relatedness are computed **natively** — a maximum-likelihood COI (inclusion-exclusion over loci with population allele frequencies, the same idea as THE REAL McCOIL's categorical method) and a frequency-adjusted IBS-moment relatedness — both validated on simulated truth. For a canonical cross-check, the pipeline also **exports** THE REAL McCOIL and Dcifer inputs (`results/export/`). Note the MLE COI, like all frequency-based COI methods, needs a **cohort** (≥ a few samples per species) to estimate allele frequencies; single-sample species fall back toward the per-locus allele count.
- In-silico validated only; wet-lab pilot pending.